Practical - 1

Practical Title: Project Initiation, Requirement Analysis, Sitemap, Wireframe, and GitHub Setup

Problem Definition: Initiate the semester-long StudentHub portal by identifying the problem scope, user roles, key modules, navigation flow, and minimum 10 pages. Create a sitemap, low-fidelity wireframe, project folder structure, README file, and GitHub repository.

Theory

1. What is URL and parts of URL?

A URL (Uniform Resource Locator) is basically a web page's internet address. It tells your browser exactly where to go to find a specific site or file on the web.

A typical URL looks like this: https://www.example.com:443/pages/about.html?user=1#team

Here are its main parts:

- **Protocol (https://):** The set of rules the browser uses to talk to the server securely.
- **Domain (example.com):** The actual name of the website you're visiting.
- **Port (:443):** The specific visual gate or port on the server (usually hidden because 443 is default for HTTPS).
- **Path (/pages/about.html):** The exact location or file path on the server.
- **Query parameters (?user=1):** Extra information sent to filter or pass data to the page.
- **Fragment (#team):** Points to a specific spot or section on that webpage so it scrolls down automatically.

References:

- MDN Web Docs - What is a URL?: https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_is_a_URL
- W3C Naming & Addressing: https://www.w3.org/Addressing/

2. How does HTML file process in web browser?

When you open an HTML file, the browser processes it in a few basic steps:

1. Parsing HTML: The browser reads your raw HTML code line by line and converts it into a structural tree called the DOM (Document Object Model).

2. Loading CSS: When it encounters CSS files or styles, it reads those and builds a CSSOM (CSS Object Model).

3. Building Render Tree: It combines the DOM and CSSOM together into a 'Render Tree' that leaves out hidden tags and focuses only on what needs to be shown on screen.

4. Layout: The browser calculates where each element goes on the screen based on screen size and dimensions.

5. Painting: Finally, it paints the actual pixels (colors, text, images, borders) onto your screen.

References:

- MDN - How Browsers Work: https://developer.mozilla.org/en-US/docs/Web/Performance/How_browsers_work
- web.dev - Constructing the Object Model: https://web.dev/articles/critical-rendering-path-constructing-the-object-model

3. How will page navigation flow managed among all HTML pages?

To manage smooth navigation across multiple HTML files in a project, I set up a few core things:

- **Logical Folder Structure:** Keep index.html as the main homepage in the root folder, and keep subpages organized in clear folders so relative file paths stay clean.
- **Consistent Navigation Bar:** Include a standard <nav> element at the top of every page containing anchor links (<a href="...">) to main pages like Home, About, and Contact.
- **Relative Links:** Use relative paths (like <a href="contact.html">) so links work seamlessly whether running locally or on a server.
- **Breadcrumbs & Footers:** Put secondary links in the footer or use breadcrumb links (Home > Products > Details) so users can easily jump back to previous sections.

References:

- W3C - Page Structure & Navigation: https://www.w3.org/WAI/tutorials/page-structure/
- MDN - Creating Hyperlinks: https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML/Creating_hyperlinks

4. How will GitHub commits be maintained after each practical?

To keep my GitHub repository clean and organized after completing each practical task, I stick to this workflow:

- **Commit Regularly:** Make a commit right after completing each logical task or practical exercise rather than dumping all changes at the very end.
- **Write Clear Commit Messages:** Use brief, action-oriented messages that describe what was added or changed (e.g., git commit -m "completed practical 2 layout using flexbox" instead of vague words like updated).
- **Check Status First:** Run git status to make sure I'm only staging relevant files and ignoring junk files.
- **Standard Push Routine:** 1) git add . (Stage updated files) | 2) git commit -m "completed practical X requirements" | 3) git push origin main (Push changes to remote GitHub repo)

References:

- GitHub Docs - About Commits: https://docs.github.com/en/pull-requests/committing-changes-to-your-project/creating-and-editing-commits/about-commits
- Pro Git Book - Recording Changes: https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository

Implementation/ Output/Result

Homepage HTML Code (index.html): Below is the HTML structure implemented for the main homepage (index.html) of the StudentHub Portal:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>StudentHub Portal</title>
  <link rel="stylesheet" href="css/script.css">
</head>
<body>
  <h1>StudentHub Portal</h1>
  <p>Welcome to the StudentHub portal. This is a simple student management website.</p>
  <h2>Main Pages</h2>
  <!-- EDITED: Added class="home-links" to style links as boxes -->
  <ul class="home-links">
    <li><a href="pages/login.html">Login</a></li>
    <li><a href="pages/registration.html">Registration</a></li>
    <li><a href="pages/profile.html">Profile</a></li>
    <li><a href="pages/dashboard.html">Dashboard</a></li>
    <li><a href="pages/course.html">Course Info</a></li>
    <li><a href="pages/assignment.html">Assignments</a></li>
    <li><a href="pages/attendance.html">Attendance</a></li>
    <li><a href="pages/result.html">Results</a></li>
    <li><a href="pages/contact.html">Contact</a></li>
  </ul>
  <h2>Portal Overview</h2>
  <p>Students can log in, view their profile, check courses, assignments, attendance,
and results.</p>
</body>
</html>
```

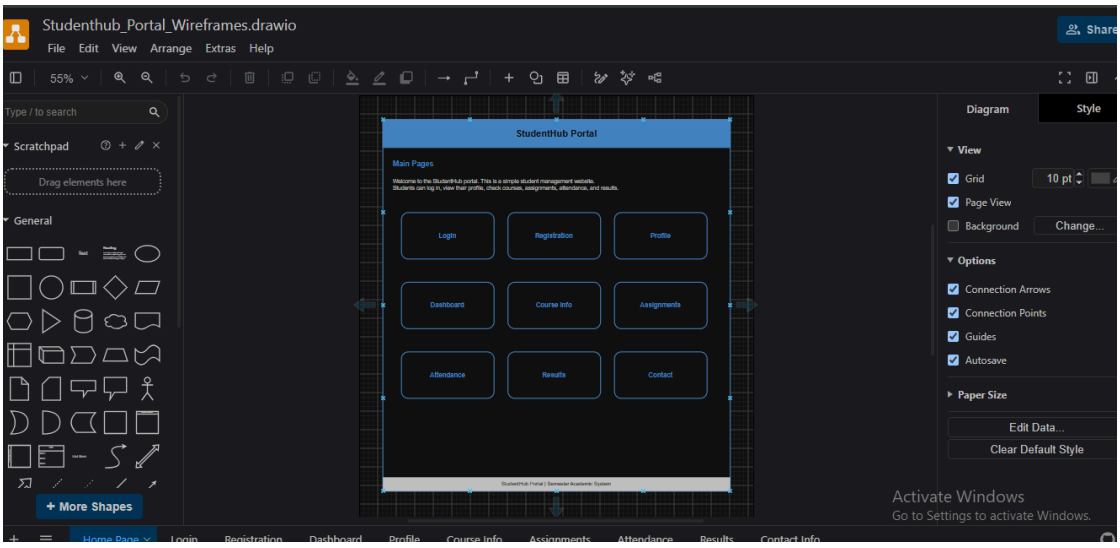
Evidence of Work (Screenshots)

Figure 1.1: Home Page Structure



Description: This screenshot displays the initial home page layout of the StudentHub portal, featuring the main heading, welcome text, navigation links list, and portal overview.

Figure 1.2: Wireframe of Home Page



Description: This screenshot displays the low-fidelity wireframe layout of the StudentHub portal home page created in Draw.io, illustrating the overall page structure and navigation card grid for Main Pages.

GitHub Repository Link:

https://github.com/VRAJ-710/studenthub-portal

Challenges Faced

Making the HTML structure was easy and sketching it out was fun too, but implementing the exact layout structure as wished was a bit difficult using pure HTML alone (for instance, rendering all navigation links as box shapes on the home page required specific CSS styling which was challenging to align perfectly in plain HTML).

Learning Outcomes

- **Sketching and Visualization:** Learned how to sketch and visually structure a website layout before writing code.
- **HTML Structure Creation:** Gained hands-on experience in building structured HTML web pages and linking them effectively.

Faculty Remarks